

Tarea 3
Image Analyzer

Google Cloud Vision API

Josefa Fernández, Felipe Fuentes, Martin Garrido

Grupo Web JFM

Demo en vivo

Mostrar las funcionalidades mínimas

- login
- consumo de API
- reactividad
- 3 funcionalidades



Distribución del trabajo

Josefa Fernández	Felipe Fuentes	Martín Garrido
• Organización del proyecto. • Documentación de ReadMe. • Investigación de API externa y uso de tema. • Estructura, diseño de pestañas, navbar, entre otros. • Reactividad, persistencia y manejo de errores y estado en UI. • Encargada de PRs y Merge conflicts.	• Log in con Firebase Auth. • Conexion a Google Vision API. • Historial con Firestore. • Deploy con google	• Integración Tailwind y estilización completa de componentes. • Mejora de la UX con feedback visuales (errores y más) • Manejo de API key, variables de entorno. • Manejo de seguridad y persistencia de datos. • Reactividad y persistencia

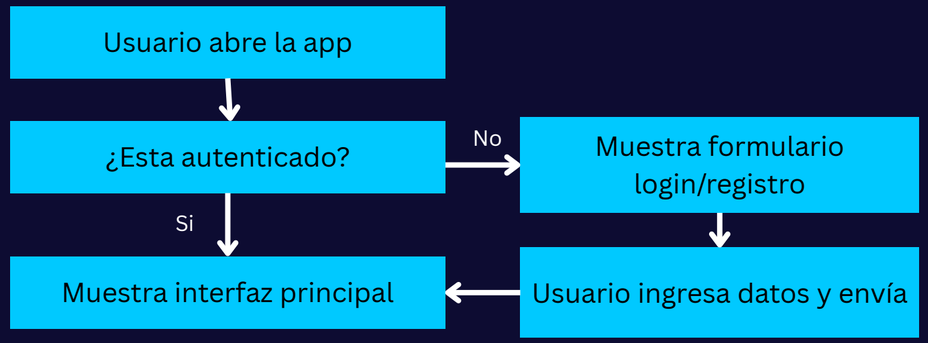
Diagramas

- Diagrama de flujo que explique el funcionamiento de cada funcionalidad

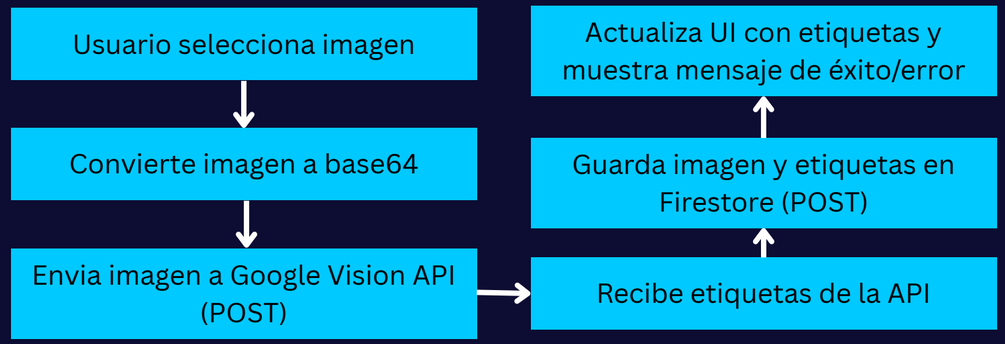


Diagramas

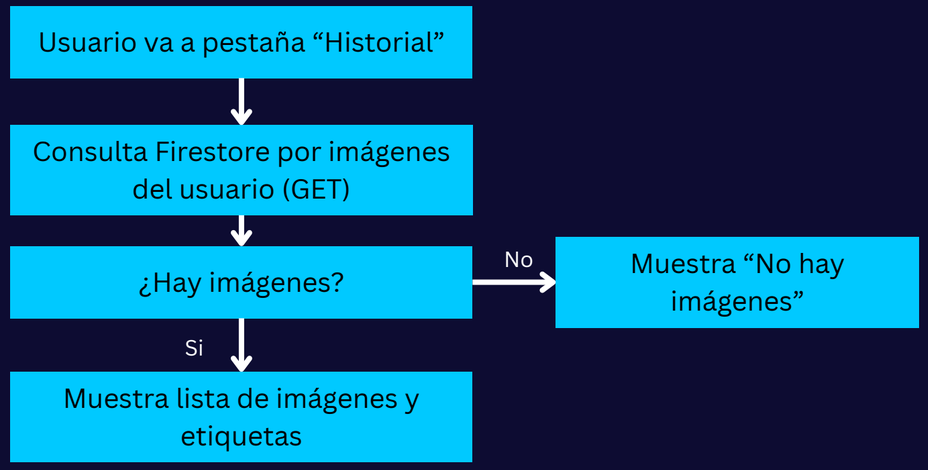
1. Inicio Sesión



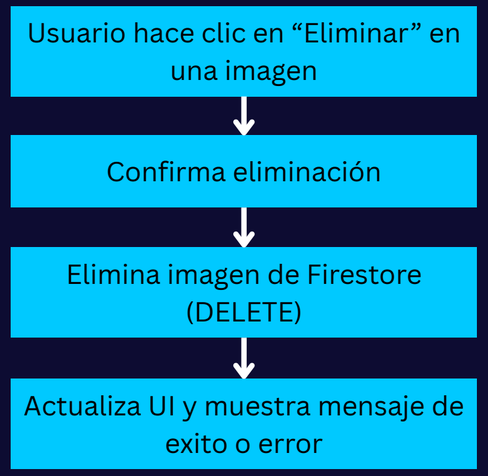
2. Subir y analizar imagen



3. Ver historial de imágenes

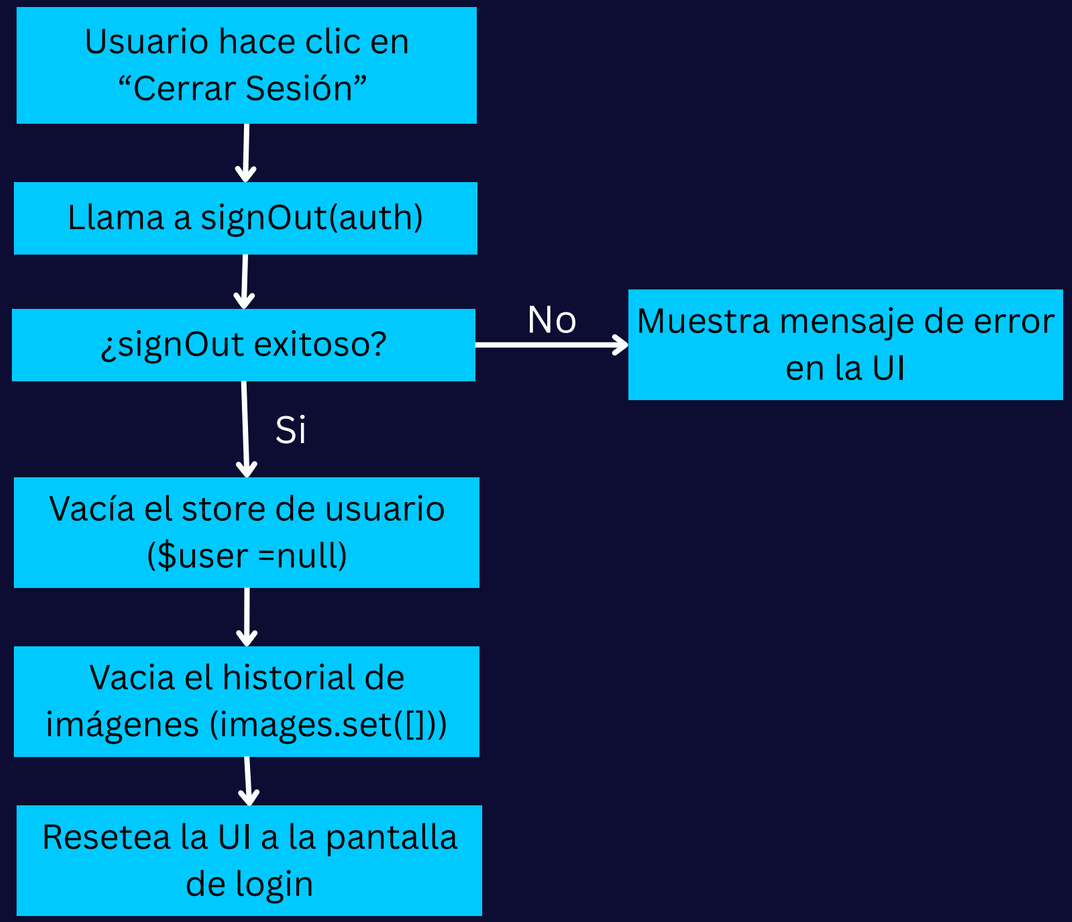


4. Eliminar Imagen del historial



Diagramas

5. Cerrar Sesión



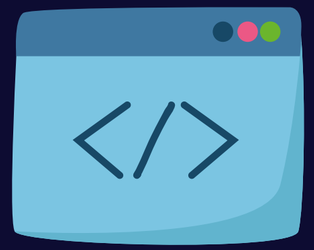
Elección de Framework: Svelte

- **Reactividad sencilla**: permite que los cambios en los datos se reflejen automáticamente en la UI sin código complejo, ideal para mostrar resultados y actualizaciones en tiempo real (como el historial de imágenes y etiquetas).

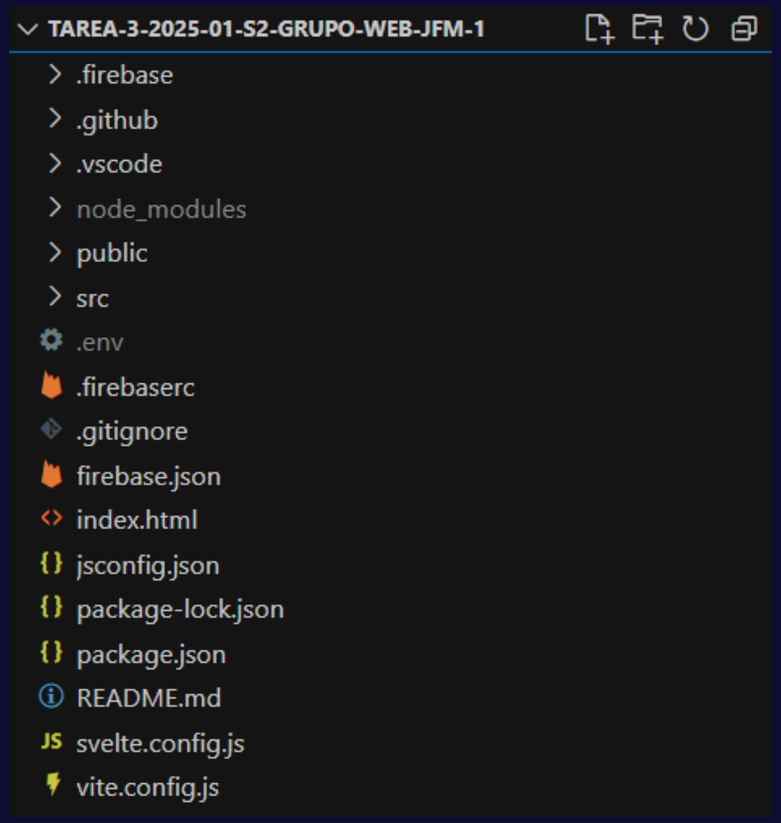
- **Componentes claros y mantenibles:** facilita la organización del código en componentes reutilizables y fáciles de entender, lo que mejora la estructura y el mantenimiento del proyecto.

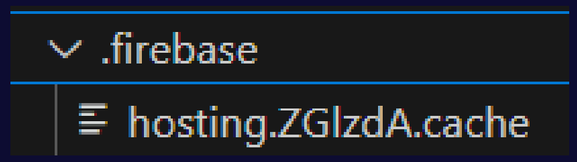
- **Rendimiento y simplicidad:** compila a JavaScript eficiente, lo que hace que la app sea rápida y ligera.

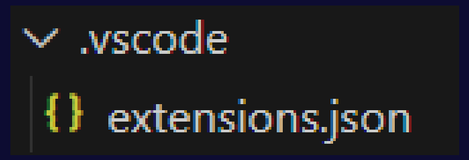




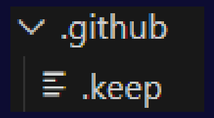
Estructura de código

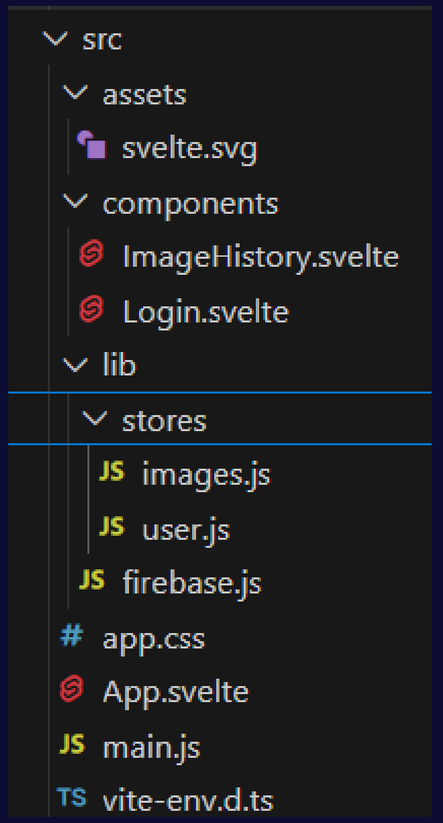












Reactividad y persistencia de datos

Reactividad:

- Cada vez que se sube, elimina o actualiza una imagen, o cambia el usuario autenticado, los componentes se actualizan automáticamente en la UI, sin necesidad de recargar la página.

Persistencia

- Los datos de usuario y las imágenes se almacenan en Firebase Authentication y Firestore. Esto permite que, aunque el usuario cierre o recargue el navegador, al volver a iniciar sesión su historial y estado se recuperan automáticamente desde la base de datos, manteniendo la lógica y el flujo de datos a través del tiempo.



Firebase
Authentication

Sin recarga de navegador: Por a la integración de Svelte y Firebase, todos los cambios (subida, borrado, análisis de imágenes, login/logout) se reflejan de forma reactiva y persistente en la interfaz, sin recargar la página

Librería de componentes: Tailwind y su diseño

- Se integró Tailwind CSS.
- Tailwind se utilizó en todos los componentes principales (App.svelte, Login.svelte, ImageHistory.svelte) para estilizar botones, listas, tarjetas y mensajes de estado.
- Con esto tiene una apariencia limpia y consistente, sin necesidad de escribir CSS personalizado desde cero.

- La interfaz es responsiva gracias a las clases utilitarias de Tailwind, que adaptan el diseño a diferentes tamaños de pantalla.
- Se usaron clases como max-w-xl mx-auto, flex, gap-4, y utilidades de espaciado y tipografía para asegurar que la UI se vea bien tanto en escritorio como en móvil.

tailwindcss



Login persistente con Firebase Auth

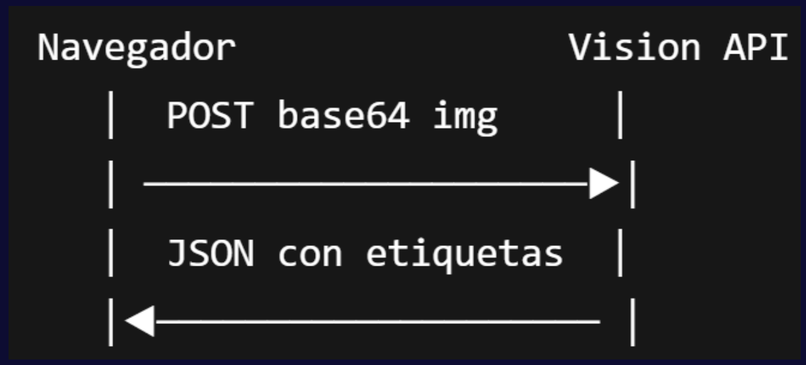
- Persistencia sin localStorage (requisito del enunciado)
- Sesión se mantiene entre pestañas y reinicios
- Logout ➔ auth.signOut()

```
const auth = getAuth(app);
await setPersistence(auth, indexedDBLocalPersistence);
await signInWithEmailAndPassword(auth, email, pwd);
```

```
[Formulario email/clave] → Firebase Auth SDK → Auth Server
                                        ↓
                                  indexedDB session
```

Integración de Google Vision API

- Imagen ➜ FileReader → base64
- fetch POST images:annotate + API_KEY
- LABEL_DETECTION, máx 5 resultados
- Respuesta JSON ➜ etiquetas + score




```
await fetch(
  `…images:annotate?key=${API_KEY}`,
  { method:"POST",
    headers:{ "Content-Type":"application/json" },
    body:JSON.stringify({
      requests:[{ image:{content:base64},
                  features:[{type:"LABEL_DETECTION",maxResults:5}] }]
```

Firestore (GET, POST, DELETE)

- Colección: images/{uid}/items → historial privado por usuario
- POST con addDoc(...) al subir imagen etiquetada
- GET con getDocs(...) al montar el historial
- DELETE con deleteDoc(...) desde el botón 🗑️
- Regla de seguridad: auth.uid == uid asegura privacidad

```
await addDoc(col(db,"images",uid,"items"), data);
const snap = await getDocs(col(db,"images",uid,"items"));
await deleteDoc(doc(db,"images",uid,"items", id));
```

Deploy con Firebase Hosting

- Usamos npm run build para generar carpeta /dist
- Firebase hace deploy del contenido estático desde /dist
- Hosting público en https://equipvalue.web.app/
- Configurado con firebase.json.

Hallazgo o Aprendizaje compartido:

- Centralización: Centralizar onAuthStateChanged es más limpio que tener múltiples listeners. Esto para evitar el error común de que la UI no refleje el estado de autenticación real. Fue el pilar de tanto la reactividad como la persistencia de datos de usuario

- Diseño: Planear el trabajo de CSS desde un principio, aún más si es con alguna librería incluida

Propuestas de funcionalidades futuras:

Funcionalidad 1: Almacenamiento Eficiente

- Descripción: Cambiar el como se estan almacenando las imágenes. Puede ser con compresión previa de estas o con Firebase Storage y manejo por URL
- Esfuerzo estimado: Medio
- Valor: Mejor rendimiento, escalabilidad y optimización de costos a largo plazo

Funcionalidad 2: Optimización del Historial

- Descripción: Actualmente si un usuario analiza muchas imágenes, cargar todo el historial puede volverse lento, incomodo y consumir muchos recursos. Podemos agregar paginación, filtros y busquedas.
- Esfuerzo estimado: Medio-Bajo
- Valor: Mejora notoria en el rendimiento, en la experiencia del usuario y mayor posibilidad de uso en el mundo real

Funcionalidad 3: Sets y etiquetas personalizadas

- Descripción: Para uso en el mundo real, la API permite la creación y “entrenamiento” de esta misma para crear etiquetas de imagenes personalizadas.
- Esfuerzo estimado: Alto
- Valor: Permite utilizar el mismo producto para distintos usos del mundo real.